Java Backend Architecture

Interview Series

Chapter 9.2
JVM Object & Heap Caching

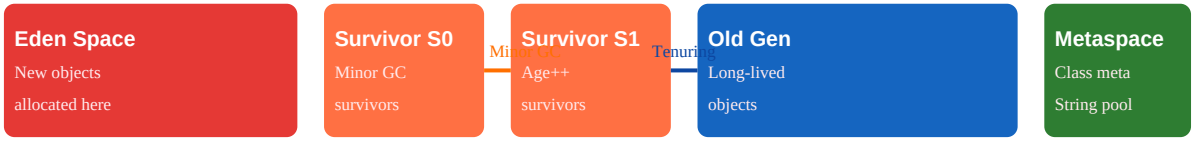
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 9.2 — JVM Object & Heap Caching

Introduction

The JVM heap itself acts as an implicit multi-level cache. The young generation (Eden + Survivors) caches short-lived objects for near-free allocation. Understanding object pooling, string interning, the Flyweight pattern, and off-heap memory allows Java developers to build GC-friendly, high-throughput caching architectures that avoid stop-the-world pauses.

JVM Heap as Implicit Cache



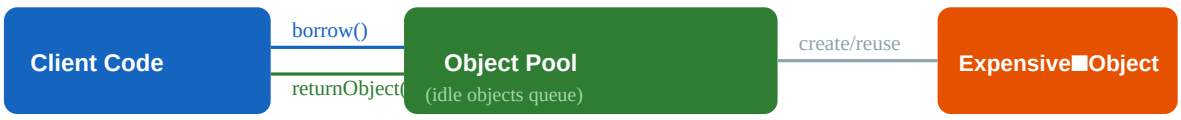
Eden = implicit object cache: allocate cheap (bump pointer), collect cheap (copy survivors only)

Object pool reuses expensive objects (DB connections, threads, ByteBuffers) across requests

Off-heap: DirectByteBuffer / Chronicle Map bypasses GC entirely — ideal for large caches (GBs)

Eden space uses **bump-pointer allocation** — allocating an object costs just a pointer increment (~10ns). Minor GC copies only surviving objects; most objects die young and are collected for free. The Old Generation holds long-lived objects (caches, singletons, connection pools). Objects that survive multiple minor GCs (default: 15 rounds) are tenured.

Object Pooling



Pool avoids: object construction cost, GC pressure, resource exhaustion

Apache Commons Pool2: GenericObjectPool with PooledObjectFactory

Max pool size = protection against resource exhaustion; min idle = warmup

```
// Apache Commons Pool2 — custom pooled object public class ByteBufferPool { private final GenericObjectPool<ByteBuffer> pool; public ByteBufferPool(int capacity, int maxSize) { pool = new GenericObjectPool<>(new PooledObjectFactory<>() { @Override public PooledObject<ByteBuffer> makeObject() { return new DefaultPooledObject<>(ByteBuffer.allocateDirect(capacity)); } @Override public void passivateObject(PooledObject<ByteBuffer> p) { p.getObject().clear(); // reset before returning to pool } @Override public void activateObject(PooledObject<ByteBuffer> p) {} @Override public boolean validateObject(PooledObject<ByteBuffer> p) { return true; } @Override public void destroyObject(PooledObject<ByteBuffer> p) {} }); GenericObjectPoolConfig<ByteBuffer> config = new GenericObjectPoolConfig<>(); config.setMaxTotal(maxSize); config.setMinIdle(2); config.setBlockWhenExhausted(true); config.setMaxWait(Duration.ofSeconds(5)); pool = new GenericObjectPool<>(factory, config); } public ByteBuffer borrow() throws Exception { return pool.borrowObject(); } public void returnBuffer(ByteBuffer buf) { pool.returnObject(buf); } }
```

String Interning

String.intern() adds a string to the JVM String Pool (in Metaspace since Java 8). Interned strings with the same content share the same object — saves memory when you have many duplicate strings (e.g., enum-like values from a DB). Caution: **intern() holds references in Metaspace** — interning arbitrary/unbounded strings causes Metaspace growth and can cause OOM. Use only for a bounded set of known strings.

```
// String Pool behaviour String a = "hello"; // in pool at compile time String b = "hello"; // same pool entry — a == b is TRUE String c = new String("hello"); // new heap object — a == c is FALSE String d = c.intern(); // returns pool entry — a == d is TRUE // Use case: interning known enum-like values from DB String status = resultSet.getString("status").intern(); // Now status comparisons can use == instead of equals() (micro-optimisation) // Metaspace monitoring: jcmd <pid> VM.native_memory summary // Or: -XX:+PrintStringTableStatistics JVM flag at shutdown // Tune string table size (default 60013 buckets): -XX:StringTableSize=131072
```

Flyweight Pattern — Integer Cache

The JVM caches **Integer objects from -128 to 127** via Integer.valueOf(). Autoboxing uses valueOf(), so boxed integers in this range are cached — the same object is returned every time. Outside this range, a new object is created. This is the Flyweight pattern: shared immutable objects for frequently-used values.

```
// Integer cache: -128 to 127 Integer a = 100; Integer b = 100; System.out.println(a == b); // true — same cached object Integer x = 200; Integer y = 200; System.out.println(x == y); // false — different heap objects // Always use equals() for Integer comparison! // Other JVM caches: // Boolean.TRUE / Boolean.FALSE // Byte: all 256 values cached // Short: -128 to 127 // Long: -128 to 127 // Character: 0 to 127 // Extend Integer cache range (JVM flag): -XX:AutoBoxCacheMax=1000 // cache Integer 0..1000
```

Off-Heap Memory

JVM Heap (On-Heap)
GC managed • fast allocation
GC pauses • max ~32GB practical

Off-Heap (Direct Memory)
No GC • TBs possible • manual free
DirectByteBuffer, Chronicle Map

Use off-heap for: large read-heavy caches (100GB+), latency-sensitive (no GC pauses), network I/O buffers

Risks: memory leaks (no GC fallback), complex lifecycle, Cleaner/PhantomReference for deallocation

Chronicle Map: persisted off-heap HashMap — survives JVM restart if memory-mapped to file

```
// DirectByteBuffer — off-heap, GC-managed deallocation via Cleaner ByteBuffer direct = ByteBuffer.allocateDirect(1024 * 1024 * 100); // 100MB // Freed when ByteBuffer is GC'd — but relies on GC to trigger Cleaner // Force cleanup: ((DirectBuffer) direct).cleaner().clean(); // Chronicle Map — persistent off-heap key-value store ChronicleMap<String, UserProfile> userCache = ChronicleMap .of(String.class, UserProfile.class) .name("user-cache") .entries(1_000_000) .averageKeySize(20) .averageValueSize(200) .createOrRecoverPersistedTo(new File("/tmp/user-cache.dat")); userCache.put("user:123", userProfile); UserProfile cached = userCache.get("user:123"); // Monitor direct memory usage: -XX:MaxDirectMemorySize=4g // cap total direct memory jcmd <pid> VM.native_memory summary // view breakdown
```

JVM Cache Sizing Guidelines

Cache Type	Typical Use	Size Guidance	GC Impact
Eden (Young Gen)	Short-lived objects, request-scoped	-Xmn or G1 adaptive	Minor GC only (cheap)
Old Gen cache	Long-lived app caches (Caffeine, maps)	50–70% of heap	Major GC (expensive)
Object Pool	DB connections, threads, buffers	Sized to concurrency	Minimal (reuse)

String Intern	Known bounded enum-like strings	-XX:StringTableSize	Metaspace growth
Direct Memory	I/O buffers, large read caches	-XX:MaxDirectMemorySize	None (off-heap)
Chronicle Map	Large persistent caches (GBs)	File system limited	None (off-heap)

50+ Interview Questions & Answers

Q1. How does Eden space act as an implicit cache?

Eden uses bump-pointer allocation (~10ns per object). Short-lived request-scoped objects are allocated in Eden and collected together during Minor GC. The surviving few are copied to Survivor spaces — collect-by-copy makes allocation effectively free for short-lived objects.

Q2. What is object pooling and when should you use it?

Reusing expensive-to-create objects (DB connections, threads, ByteBuffers, parsers) instead of creating/destroying them per-request. Use when: object creation is expensive (>1ms), the object is stateless or resettable, and concurrency is bounded.

Q3. What is Apache Commons Pool2?

A generic object pooling library. GenericObjectPool manages a pool of objects created by a PooledObjectFactory. Configurable: maxTotal (max pool size), minIdle (min warm objects), maxWait (borrow timeout), testOnBorrow (validate before use).

Q4. What is String interning?

String.intern() returns the canonical String from the JVM String Pool. Two interned strings with equal content return the same object reference. Enables == comparison instead of equals(). Strings in pool are stored in Metaspace (Java 8+).

Q5. What are the risks of String.intern()?

Interning arbitrary/unbounded strings causes Metaspace growth — potentially OOM. The intern pool is a hash table; poor distribution causes O(n) lookup. Use only for a known, bounded set of strings (e.g., status codes, country names).

Q6. What is the Flyweight pattern and how does the JVM use it?

Flyweight shares immutable objects to reduce memory. JVM uses it for: Integer.valueOf(-128 to 127), Boolean.TRUE/FALSE, all Byte values, String pool, Character (0-127). Autoboxing uses valueOf() — exploits the cache automatically.

Q7. What is the Integer cache range and how can it be extended?

Default: -128 to 127. Extend the upper bound with -XX:AutoBoxCacheMax=N (e.g., N=1000). Lower bound is always -128 (JLS mandated). Only affects Integer — not Long, Short, etc. (those always cache -128..127).

Q8. What is DirectByteBuffer?

A ByteBuffer allocated outside the JVM heap (in native OS memory). Not subject to GC — must be freed via Cleaner. Ideal for I/O operations (DMA transfer without copy), NIO channels, and large caches that should not cause GC pauses.

Q9. What is Chronicle Map?

An off-heap, persisted, concurrent HashMap implementation. Stores data in memory-mapped files — survives JVM restarts. Supports TBs of data. Zero GC impact. Used for large caches, IPC between JVM processes.

Q10. When should you use off-heap memory?

When: (1) cache size would push heap to GC-pause territory (>4GB for low-latency apps), (2) data must survive JVM restart, (3) multiple JVM processes share cache, (4) raw I/O buffer performance is critical.

Q11. What is -XX:MaxDirectMemorySize?

Caps total off-heap (direct) memory. Default: equal to -Xmx. JVM throws OOM: Direct buffer memory when exceeded. Set explicitly for services with heavy NIO usage: -XX:MaxDirectMemorySize=4g.

Q12. How do you force deallocation of a DirectByteBuffer?

((sun.nio.ch.DirectBuffer) buffer).cleaner().clean() forces the Cleaner to run and free native memory immediately. Normally deallocation happens when the ByteBuffer is GC'd — but GC may not run soon enough for large allocations.

Q13. What is the difference between heap and direct ByteBuffer performance?

Direct: better for I/O (no copy between heap and native), worse for random reads (no CPU cache benefit, off-heap access has extra indirection). Heap: better for in-process computation, GC managed, simpler lifecycle. Use direct for I/O, heap for computation.

Q14. What is tenuring threshold in JVM GC?

-XX:MaxTenuringThreshold (default: 15). Objects surviving this many Minor GCs are promoted to Old Gen. Lower value: earlier promotion (reduces Minor GC copy work). Higher value: longer in Young Gen (fewer Major GCs).

Q15. What is the G1GC humongous object?

Objects larger than 50% of G1 region size are 'humongous' — allocated directly in Old Gen regions (bypassing Eden). They are expensive because they cause fragmentation and trigger Full GC more easily. Avoid humongous allocations in hot paths.

Q16. How do you monitor String pool usage?

-XX:+PrintStringTableStatistics at JVM shutdown. Or: jcmd VM.stringtable. Shows bucket count, size, collisions, and total strings interned.

Q17. What is Escape Analysis in JVM?

JIT optimization: if an object doesn't escape the current method/thread, the JIT may allocate it on the stack (stack allocation) instead of the heap — zero GC overhead. Enabled by default: -XX:+DoEscapeAnalysis.

Q18. What is scalar replacement?

JIT optimization related to escape analysis: if an object doesn't escape, the JIT replaces it with its individual fields as stack variables — avoiding object creation entirely. Common for small value holders (Point, Pair).

Q19. What is the WeakHashMap?

A HashMap where keys are held with WeakReferences — the GC can collect entries when the key has no other strong references. Useful for memory-sensitive caches where entries should auto-expire when key is no longer used.

Q20. What is SoftReference and when is it useful for caching?

SoftReference objects are collected only when JVM needs memory (OOM pressure). Used for in-memory caches that should grow as much as possible but won't cause OOM. Caffeine and Guava use SoftValues for size-based eviction.

Q21. What is the difference between WeakReference and SoftReference?

WeakReference: collected on next GC cycle when no strong references exist. SoftReference: collected only under memory pressure (OOM imminent). For caching: SoftReference is more useful (lives longer);

WeakReference for canonicalisation maps.

Q22. What is a ReferenceQueue?

A queue populated by the GC when a Reference (Weak/Soft/Phantom) is collected. Used for cleanup: register a Cleaner with ReferenceQueue to free resources when an object is GC'd (DirectByteBuffer cleanup, off-heap deallocation).

Q23. How does Caffeine handle soft references?

Caffeine.newBuilder().softValues() wraps cached values in SoftReferences. Under memory pressure, GC can evict entries. Less deterministic than size-based eviction — prefer maximumSize + expireAfterWrite for predictable behaviour.

Q24. What is the object header overhead in Java?

Every Java object has: mark word (8 bytes — GC age, lock state, hash code) + class pointer (4 bytes compressed oops, or 8 bytes). Total: 12-16 bytes header. Minimum object size: 16 bytes (padded to 8-byte alignment).

Q25. How do you reduce object creation overhead?

Use primitive arrays instead of object arrays, primitive types instead of boxed, value objects (records), object pooling for expensive objects, StringBuilder instead of String concatenation, avoid autoboxing in hot loops.